

Курс: Углубленное машинное обеспечение и искусственный интеллект в Python

Практическое задание: Проектирование и разработка AI-агентов

Цель работы: Применить на практике концепцию агентного ИИ (Agentic AI), научиться использовать локальные языковые модели через Ollama, освоить базовые инструменты оркестрации фреймворка LangChain (или LangGraph) и построить автономного текстового агента в среде Codex, способного использовать внешние инструменты (Tools) для решения пользовательских задач.

Шаг 1. Настройка агентской среды и стека

Для реализации агента вам потребуется развернуть локальную среду разработки.

Убедитесь, что у вас установлены и настроены следующие компоненты:

1. **Ollama:** Утилита для локального запуска больших языковых моделей. Для работы рекомендуется загрузить легковесную, но способную к вызову инструментов (tool calling) модель, например llama3 или mistral.
2. **Среда Codex / Jupyter:** Интерактивная среда для запуска и отладки агентских сессий.
3. **Библиотеки:** langchain, langchain-community, ollama.

```
# Базовая проверка доступности локальной LLM через LangChain
from langchain_community.llms import Ollama
```

```
llm = Ollama(model="llama3")
# Убедитесь, что модель отвечает на тестовый запрос
```

Шаг 2. Выбор сценария и проектирование агента

Вам необходимо спроектировать и собрать **одного** простого агента на выбор. Агент должен обладать системным промптом (ролью), понимать намерения пользователя и

уметь вызывать как минимум **два простых инструмента** (функции Python, обернутые в Tools).

Варианты простых агентов для реализации:

- **Вариант А: Агент-аналитик умного дома (Smart Home Agent)**
 - *Роль:* Дворецкий, управляющий девайсами.
 - *Инструменты:* Функция проверки текущей «температуры» (возвращает случайное число или константу) и функция «включения/выключения света» (изменяет глобальную переменную-флаг и выводит лог в консоль).
 - *Задача:* Пользователь пишет: «Мне жарко, и в комнате слишком темно». Агент должен последовательно вызвать нужные инструменты.
- **Вариант В: Агент-помощник по локальным файлам (File Commander Agent)**
 - *Роль:* Ассистент по работе с документами.
 - *Инструменты:* Функция чтения структуры текущей директории (список файлов) и функция подсчета количества слов в указанном текстовом файле.
 - *Задача:* Пользователь просит: «Найди в этой папке отчет и скажи, сколько в нем текста».
- **Вариант С: Финансовый калькулятор-ассистент (Finance Agent)**
 - *Роль:* Быстрый калькулятор с доступом к базовой математике и конвертации.
 - *Инструменты:* Функция точного расчета сложных процентов и функция конвертации валюты по фиксированному актуальному курсу.
 - *Задача:* Пользователь спрашивает: «Сколько я получу через год, если вложу 10 000 рублей под 10% годовых, и сколько это будет в долларах?».

Шаг 3. Создание инструментов (Tools) и сборка агента

1. Напишите стандартные функции на Python, реализующие логику выбранного варианта.
2. Оберните их в инструменты LangChain с помощью декоратора `@tool`. Обязательно напишите подробный docstring к каждой функции — именно по нему LLM понимает, зачем нужен этот инструмент.
3. Инициализируйте агента, передав ему модель, набор инструментов и системный промпт.

```
from langchain.agents import AgentExecutor,
create_tool_calling_agent
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.tools import tool
```

```
# Пример создания простейшего инструмента
@tool
```

```
def get_weather_forecast(location: str) -> str:
    """Возвращает текущую погоду для указанного города
(location)."""
    # Здесь логика симуляции или вызова API
    return f"В городе {location} сейчас +22°C, солнечно."

# Настройка промпта агента
prompt = ChatPromptTemplate.from_messages([
    ("system", "Ты — полезный ИИ-ассистент, использующий
инструменты для точных ответов."),
    ("placeholder", "{chat_history}"),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])

# Сборка агента и исполнителя (AgentExecutor)
# (Заполните этот блок самостоятельно, используя документацию
LangChain)
```

Шаг 4. Тестирование и оценка автономности

1. Напишите сценарий из 3–4 разнообразных запросов к вашему агенту. Среди запросов должен быть как минимум один «сложный», требующий поочередного вызова обоих инструментов, и один запрос «не по теме» (где инструменты применять не нужно).
2. Настройте вывод агента в режиме verbose=True, чтобы в консоли Codex наглядно отображался ход мыслей модели (Thought -> Action -> Observation -> Final Answer).
3. Проанализируйте логи работы: всегда ли локальная модель безошибочно извлекает аргументы для функций и правильно ли она интерпретирует результаты работы инструментов для финального ответа пользователю. Сделайте краткий вывод.